

On Momentum Acceleration for Randomized Coordinate Descent in Matrix Completion

Matthew Callahan¹, Trung Vu², and Raviv Raich¹

¹School of EECS, Oregon State University, Corvallis, OR 97331-5501, USA

² Department of CSEE, University of Maryland, Baltimore County, MD 21250-0002, USA

{callamat, raich}@oregonstate.edu trungvv@umbc.edu

Abstract—Matrix completion plays an important role in machine learning and signal processing, with applications ranging from recommender systems to image inpainting. Many approaches have been considered to solve the problem and some offer computationally efficient solutions. In particular, a highly-efficient random coordinate descent approach reduces the per-epoch computation dramatically. This paper is concerned with further improvement of computational efficiency to expand the range of problem sizes and condition that can be solved. Momentum acceleration is a well-known method to improve the efficiency of iterative algorithms, but applying it to random coordinate descent methods without increasing the computational complexity is non-trivial. To address this challenge, we introduce a momentum-accelerated randomized coordinate descent for matrix completion approach that does not increase computational complexity by accelerating at the level of epochs. Additionally, we propose an analysis-driven, tuning-free method for step size selection. To that end, we offer a convergence rate analysis for the algorithm. Using numerical evaluations, we demonstrate the competitiveness of the method and verify the theoretical analysis.

Index Terms—coordinate descent, asymptotic convergence analysis, matrix completion, randomized methods.

I. INTRODUCTION

Matrix completion addresses the recovery of a rank- r matrix $\mathbf{M} \in \mathbb{R}^{m \times n}$ from a subset of s entries, $\Omega = \{(i, j) \in [m] \times [n] \mid M_{ij} \text{ is observed}\}$. One application is recommendation systems in which the objective is to estimate the full set of ratings for every user for every product, $\mathbf{M}$, using only a small subset of user-product ratings, Ω [1]. Numerous other additional applications in signal processing also apply including image inpainting [2] and device localization [3]. Using a rank- r factorization, the matrix completion problem can be reformulated as finding an $m \times r$ matrix $\mathbf{A}$ and an $n \times r$ matrix $\mathbf{B}$ such that $(\mathbf{AB}^\top)_{ij} = M_{ij}$ for all $(i, j) \in \Omega$. The following optimization is used to formalize matrix completion:

$$\min_{\mathbf{A} \in \mathbb{R}^{m \times r}, \mathbf{B} \in \mathbb{R}^{n \times r}} \frac{1}{2} \|\mathcal{P}_\Omega(\mathbf{AB}^\top - \mathbf{M})\|_F^2, \quad (1)$$

where the projection $\mathcal{P}_\Omega : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{m \times n}$ is defined as

$$[\mathcal{P}_\Omega(\mathbf{X})]_{ij} = \begin{cases} X_{ij} & (i, j) \in \Omega, \\ 0 & (i, j) \notin \Omega. \end{cases} \quad (2)$$

There exists a long line of work focused on first-order iterative methods for low-rank matrix completion, including gradient descent and its variations [4]–[6], iterative hard-thresholding [7], [8], and alternating minimization [9], [10]. Recently, we proposed a simple yet competitive approach for solving matrix completion via random coordinate descent (RCD) [11]. A theoretical rate of convergence analysis was performed and confirmed via empirical evaluation. Moreover, the approach benefits from a low per-epoch computational complexity of $O(sr)$. However, to extend the applicability of the

approach, we are interested in finding methods that can improve the rate of convergence without increasing the per-epoch computational complexity.

The acceleration of RCD methods has been studied in [12]–[14]. These methods apply acceleration with the update of each coordinate. This is efficient under the assumption that calculating one entry of the gradient is on the order of the number of coordinates. However, for matrix completion via factorization, calculating the gradient for a single factor entry is of a lower order than the number of unknown entries, since only one column of $\mathbf{A}$ or of $\mathbf{B}$ is needed. Hence, a momentum acceleration step increases the computational complexity order of a single coordinate update, and care needs to be taken as to how often this step is performed.

Contributions To address the aforementioned challenge, we introduce a per-epoch momentum acceleration of factor-based randomized coordinate descent for matrix completion. Additionally, we provide a rate of convergence analysis on the mean error vector, which we confirm using numerical experiments. Using this analysis we introduce a method for determining the momentum coefficient a priori, rendering our method tuning-free. Moreover, we demonstrate empirically that the proposed method outperforms state-of-the-art methods, achieving a superior convergence rate while maintaining the same per-epoch computational efficiency as coordinate descent.

Notation. Throughout the paper, we use the notations $\|\cdot\|_F$ and $\|\cdot\|$ to denote the Frobenius norm of a matrix and the Euclidean norm of a vector, respectively. Boldfaced symbols are reserved for vectors and matrices, while the elements of a vector/matrix are unbold. In addition, $\mathbf{I}_n$ denotes the $n \times n$ identity matrix. The notation $\mathbf{e}_i^{(n)}$ denotes the i th vector in the natural basis of $\mathbb{R}^n$. We use $\otimes$ to denote the Kronecker product between two matrices and $\text{vec}(\mathbf{X})$ to denote the vectorization of $\mathbf{X}$ by stacking its columns on top of one another. We use $\mathbf{A}^{(k)}$ to denote iterations of internal algorithms, and $\mathbf{A}_k$ to denote iterations of external loops.

II. BACKGROUND

To set the ground for understanding the momentum-based acceleration of RCD for matrix completion, we begin with a brief overview of momentum acceleration for fixed-point iterations and proceed with the RCD algorithm and its convergence rate analysis.

A. Momentum-based acceleration for fixed-point iterations

Consider a real-valued vector function $\mathbf{f}$ of a vector variable and given a point $\mathbf{x}_0$ in the domain of $\mathbf{f}$, the fixed-point iteration is given by

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k), \quad \text{for } k = 0, 1, 2, \dots,$$

which yields a sequence of vectors $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots$. If $\mathbf{f}$ is a contraction map, this sequence converges to a fixed point $\mathbf{x}^*$ such that

This work was partially funded by the Consortium for Nuclear Forensics under Department of Energy National Nuclear Security Administration award number DE-NA0004142

$\mathbf{f}(\mathbf{x}^*) = \mathbf{x}^*$. If $\mathbf{f}$ is twice differentiable, then one can derive the update equation for the error $\epsilon_k = \mathbf{x}_k - \mathbf{x}^*$ as a recursion

$$\epsilon_{k+1} = \mathbf{T}\epsilon_k + \mathbf{q}(\epsilon_k), \quad (3)$$

where $\mathbf{T} = \nabla \mathbf{f}(\mathbf{x}^*)$ and $\mathbf{q}$ is the residual linear approximation error satisfying $\|\mathbf{q}(\epsilon)\| \leq q\|\epsilon\|^2$ for any $\epsilon \in [15]$.

In [16], Vu and Raich showed that if $\mathbf{T}$ is diagonalizable, then for arbitrarily small $\bar{\epsilon} > 0$ it holds that $\|\epsilon_k\| < \bar{\epsilon}$, $\forall k \geq N$, where N is the smallest integer satisfying

$$N \geq \frac{\log(1/\bar{\epsilon})}{\log(1/\rho(\mathbf{T}))} + c \left(\rho(\mathbf{T}), \frac{q\|\epsilon_0\|}{1 - \rho(\mathbf{T})} \right), \quad (4)$$

where $0 \leq \rho(\mathbf{T}) < 1$ is the spectral radius of matrix $\mathbf{T}$, $q = \sup_{\|\delta\| \leq \|\epsilon_0\|} \frac{\|\mathbf{q}(\delta)\|}{\|\delta\|^2}$, and $c(\cdot)$ is a function independent of $\bar{\epsilon}$. This suggests that the error norm decreases exponentially at rate $\rho = \rho(\mathbf{T})$ as $\|\epsilon_k\| \leq C\rho^k$.

To accelerate the convergence of $\mathbf{x}_k$ towards $\mathbf{x}_*$, a momentum-accelerated version of the fixed-point iteration can be considered

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k) + \beta(\mathbf{x}_k - \mathbf{x}_{k-1}).$$

The resulting error update equation is then

$$\epsilon_{k+1} = \mathbf{T}\epsilon_k + \beta(\epsilon_k - \epsilon_{k-1}) + \mathbf{q}(\epsilon_k). \quad (5)$$

By stacking ϵ_{k+1} and ϵ_k together, this equation can be rewritten as

$$\begin{bmatrix} \epsilon_{k+1} \\ \epsilon_k \end{bmatrix} = \mathbf{H}(\beta) \begin{bmatrix} \epsilon_k \\ \epsilon_{k-1} \end{bmatrix} + \begin{bmatrix} \mathbf{q}(\epsilon_k) \\ \mathbf{q}(\epsilon_{k-1}) \end{bmatrix}, \quad \mathbf{H}(\beta) = \begin{bmatrix} \mathbf{T} + \beta\mathbf{I} & -\beta\mathbf{I} \\ \mathbf{I} & \mathbf{0} \end{bmatrix}.$$

Then, from (4), the number of iterations N required to achieve an error of $\bar{\epsilon}$ is the smallest integer N such that

$$N \geq \frac{\log(1/\bar{\epsilon})}{\log(1/\rho(\mathbf{H}(\beta)))} + c \left(\rho(\mathbf{H}(\beta)), \frac{q\|\epsilon_0\|}{1 - \rho(\mathbf{H}(\beta))} \right),$$

for which the rate is $\rho = \rho(\mathbf{H}(\beta))$. If $\mathbf{T}$ has all real eigenvalues between 0 and 1, then by setting β to the optimal value $\beta^* = (1 - \sqrt{1 - \rho(\mathbf{T})})^2$, the fastest rate of convergence (i.e., the smallest ρ) can be achieved and is $\rho(\mathbf{H}(\beta^*)) = \sqrt{\beta^*}$ [17].

The rate of slowly converging unaccelerated fixed-point algorithms can be expressed $\rho(\mathbf{T}) = \rho(\mathbf{I} - \mathbf{Q}) = 1 - \delta$, where δ is assumed small (i.e., $\delta \ll 1$). The number of steps to achieve $\bar{\epsilon}$ error is proportional (up to an additive constant) to $1/\log(1/\rho) \approx 1/\delta$. On the other hand, the rate for the accelerated algorithm with optimal step size is $\rho(\mathbf{H}) \approx 1 - \sqrt{\delta}$ and consequently the number of steps would be proportional to $1/\sqrt{\delta}$. This suggests that momentum acceleration with an optimal step-size selection can reduce the number of iterations to the square root of the number of iterations in the unaccelerated case.

B. Randomized coordinate descent for matrix completion

1) *Algorithm*: In [11], a regularization-free randomized coordinate descent algorithm on the factors is presented. To handle factor ambiguity, a refactorization is included. The algorithm performs iteration k by selecting a coordinate uniformly from the set $\{A_{11}, \dots, A_{mr}, B_{11}, \dots, B_{nr}\}$ and minimizing the error (1) with respect to that coordinate holding the remaining coordinates fixed. During an entry update, both the single entry of $\mathbf{A}^{(k)}$ or $\mathbf{B}^{(k)}$ and the corresponding row or column of the residual error matrix $\mathbf{R}^{(k)} = \mathcal{P}_\Omega(\mathbf{A}^{(k)}(\mathbf{B}^{(k)})^\top - \mathbf{M})$ are updated as in Algorithm 1.

The rate of convergence of the error between $\mathbf{X}^{(k)} = \mathbf{A}^{(k)}\mathbf{B}^{(k)\top}$ and $\mathbf{M}$ depends on the normalized columns of $\mathbf{A}$ and $\mathbf{B}$ at the solution. Hence, one way to maintain a predictable rate is to construct a unique factorization (up to permutation/sign). This is the approach taken by [11]. Given arbitrary factors $\mathbf{A}$ and $\mathbf{B}$, new factors $\tilde{\mathbf{A}}$ and

Algorithm 1: One epoch of Randomize Coordinate Descent (RCD) for Matrix Completion modified from [11]

Input: Initial value $\mathbf{A} \in \mathbb{R}^{m \times r}$ and $\mathbf{B} \in \mathbb{R}^{n \times r}$, $\mathcal{P}_\Omega(\mathbf{M})$, Ω , direction vector $\mathbf{s} \in \mathbb{R}^m$

Output: Normalized factors $\tilde{\mathbf{A}} \in \mathbb{R}^{m \times r}$ and $\tilde{\mathbf{B}} \in \mathbb{R}^{n \times r}$

- 1: $\mathbf{R} \leftarrow \mathcal{P}_\Omega(\mathbf{A}\mathbf{B}^\top - \mathbf{M})$ for all $(i, j) \in \Omega$
- 2: **for** $k = 1, 2, \dots, (m+n)r$ **do**
- 3: Sample $p \in \{1, \dots, m+n\}$ uniformly at random
- 4: **if** $p \in [1, m]$ **then** $\triangleright$ Perform the A-update
- 5: Sample $(i, j) \in \{1, \dots, m\} \times \{1, \dots, r\}$ uniformly
- 6: $\gamma = \frac{\sum_{l:(i,l) \in \Omega} R_{il} B_{lj}}{\sum_{l:(i,l) \in \Omega} B_{lj}^2}$ $\triangleright O(s/m)$
- 7: $A_{ij} \leftarrow A_{ij} - \gamma$ $\triangleright O(1)$
- 8: **for** $h : (i, h) \in \Omega$ **do** $\triangleright O(s/m)$
- 9: $R_{ih} \leftarrow R_{ih} - \gamma B_{hj}$
- 10: **else** $\triangleright$ Perform the B-update
- 11: Sample $(i, j) \in \{1, \dots, n\} \times \{1, \dots, r\}$ uniformly
- 12: $\gamma = \frac{\sum_{l:(l,i) \in \Omega} R_{li} A_{lj}}{\sum_{l:(l,i) \in \Omega} A_{lj}^2}$ $\triangleright O(s/n)$
- 13: $B_{ij} \leftarrow B_{ij} - \gamma$ $\triangleright O(1)$
- 14: **for** $h : (h, i) \in \Omega$ **do** $\triangleright O(s/n)$
- 15: $R_{hi} \leftarrow R_{hi} - \gamma A_{hj}$
- 16: $(\tilde{\mathbf{A}}, \tilde{\mathbf{B}}) = \text{Refactor}(\mathbf{A}, \mathbf{B}, \mathbf{s})$ $\triangleright O((m+n+r)r^2)$

$\tilde{\mathbf{B}}$ are produced such that $\tilde{\mathbf{A}}\tilde{\mathbf{B}}^\top = \mathbf{A}\mathbf{B}^\top$ and $\tilde{\mathbf{A}}^\top\tilde{\mathbf{A}} = \tilde{\mathbf{B}}^\top\tilde{\mathbf{B}} = \mathbf{\Sigma}$ where $\mathbf{\Sigma}$ is the matrix of singular values of $\mathbf{X}$. This procedure follows Lines 1-7 of Algorithm 2. Assuming distinct values of $\mathbf{\Sigma}$, this procedure produces unique factors $\tilde{\mathbf{A}}$ and $\tilde{\mathbf{B}}$ up to a consistent permutation and/or sign change of paired columns. This refactorization procedure is performed every $r(n+m)$ element updates, namely, an epoch. This allows the computational complexity of the entire algorithm to be $O(sr)$ per epoch. Recall that $s = |\Omega|$ is the number of known entries.

2) *Error Analysis*: In [11], an error analysis is presented on $\delta^{(k)} = (\mathbf{Z}^\top \mathbf{S} \mathbf{S}^\top \mathbf{Z})^{1/2} \mathbf{Z}^\top \text{vec}(\mathbf{A}^{(k)}(\mathbf{B}^{(k)})^\top - \mathbf{M})$, a projected version of the error between $\mathbf{X}^{(k)} = \mathbf{A}^{(k)}\mathbf{B}^{(k)\top}$ and $\mathbf{M}$. Specifically, it is shown that the error follows

$$\delta^{(k)} = \mathbf{T}^{(k)} \delta^{(k-1)} + \mathbf{q}(\delta^{(k-1)}),$$

where $\mathbf{q}(\delta) \leq c\|\delta\|^2$ while $\mathbf{T}^{(k)}$ is a random matrix whose mean is given by $\mathbf{T} = \mathbb{E}[\mathbf{T}^{(k)}] = \mathbf{I} - \mathbf{Q}$. The analysis yields

$$\begin{aligned} \mathbf{Q} &= \frac{1}{(m+n)r} \sum_{i=1}^m \sum_{j=1}^r \frac{\mathbf{G}^{1/2} \mathbf{Z}^\top (\mathbf{v}_j \otimes \mathbf{e}_i^{(m)}) (\mathbf{v}_j \otimes \mathbf{e}_i^{(m)})^\top \mathbf{Z} \mathbf{G}^{1/2}}{\|\mathbf{G}^{1/2} \mathbf{Z}^\top (\mathbf{v}_j \otimes \mathbf{e}_i^{(m)})\|^2} \\ &+ \frac{1}{(m+n)r} \sum_{i=1}^n \sum_{j=1}^r \frac{\mathbf{G}^{1/2} \mathbf{Z}^\top (\mathbf{e}_i^{(n)} \otimes \mathbf{u}_j) (\mathbf{e}_i^{(n)} \otimes \mathbf{u}_j)^\top \mathbf{Z} \mathbf{G}^{1/2}}{\|\mathbf{G}^{1/2} \mathbf{Z}^\top (\mathbf{e}_i^{(n)} \otimes \mathbf{u}_j)\|^2}, \end{aligned} \quad (6)$$

with $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ the reduced singular value decomposition of $\mathbf{M}$, $\mathbf{Z} \in \mathbb{R}^{mn \times (m+n-r)r}$ an orthonormal basis of $\mathbf{I}_{mn} - \mathbf{P}_{V_\perp} \otimes \mathbf{P}_{U_\perp}$, $\mathbf{u}_j = \mathbf{U} \mathbf{e}_j^{(r)}$, $\mathbf{v}_j = \mathbf{V} \mathbf{e}_j^{(r)}$, $\mathbf{S}$ a selection matrix such that $\mathbf{S} \mathbf{S}^\top \text{vec}(\mathbf{X}) = \text{vec}(\mathcal{P}_\Omega(\mathbf{X}))$, and $\mathbf{G} = \mathbf{Z}^\top \mathbf{S} \mathbf{S}^\top \mathbf{Z}$. The expected error based on the previous iteration is then

$$\mathbb{E}[\delta^{(k+1)}] = (\mathbf{I} - \mathbf{Q}) \mathbb{E}[\delta^{(k)}] + O(\|\delta^{(k)}\|^2).$$

However, as in [18], since the index is selected i.i.d. between iterations, the above equation can be applied repeatedly to obtain the expected error after t epochs as

$$\mathbb{E}[\delta^{(k+(n+m)rt)}] = (\mathbf{I} - \mathbf{Q})^{(n+m)rt} \mathbb{E}[\delta^{(k)}] + O(\|\delta^{(k)}\|^2). \quad (7)$$

III. PROPOSED ALGORITHM

In this section, we introduce momentum acceleration to matrix completion by random coordinate descent. Our proposed routine is as follows:

- 1) Initialize $\mathbf{A}_0, \mathbf{B}_0$
- 2) Set $\mathbf{A}_1 = \mathbf{A}_0, \mathbf{B}_1 = \mathbf{B}_0$
- 3) for $k = 1, 2, \dots$

- a) Generate $(\tilde{\mathbf{A}}_k, \tilde{\mathbf{B}}_k)$ from iterated RCD:

$$(\tilde{\mathbf{A}}_k, \tilde{\mathbf{B}}_k) = \underbrace{\text{RCD}(\text{RCD}(\dots \text{RCD}(\mathbf{A}_k, \mathbf{B}_k)))}_{t \text{ nested functions}}$$

- b) Update the factors as follows:

$$\mathbf{A}_{k+1} = \tilde{\mathbf{A}}_k + \beta(\mathbf{A}_k - \mathbf{A}_{k-1}), \quad (8)$$

$$\mathbf{B}_{k+1} = \tilde{\mathbf{B}}_k + \beta(\mathbf{B}_k - \mathbf{B}_{k-1}). \quad (9)$$

A pseudo-code for the routine is provided in Algorithm 3. Unlike other random coordinate descent acceleration methods in the literature, we delay the acceleration step to occur after t epochs in order to preserve the computational complexity.

Remark 1. Applying acceleration more frequently than once per epoch increases the complexity. Accelerating individual coordinates as in [14], increases the per-epoch computational complexity from $O(sr)$ to $O(sr) + O((n+m)^2 r^2)$ since the acceleration operation is $O((n+m)r)$ while each coordinate update is $O(\frac{s}{n+m})$.

Modified refactorization Due to the refactorization procedure from [11], columns of $\mathbf{A}_k$ are ordered and scaled versions of the left singular vectors of $\mathbf{X}_k$. In our analysis, we expect the column subspace to change gradually from one iteration to the next. However, a sign ambiguity issue may lead to a column of $\mathbf{A}_k$ to change rapidly, meaning $\mathbf{A}_k$ and $\mathbf{X}_k$ would change at different rates, and the algorithm could fail. Hence, we modify the refactorization procedure to enforce a consistent sign in the factors. First, we perform the procedure in [11] (Lines 1-7 of Algorithm 2). Let the factors produced by this process be

$$(\hat{\mathbf{A}}, \hat{\mathbf{B}}) = \text{Refactor}(\mathbf{A}, \mathbf{B}) \quad (\text{Lines 1-7}).$$

Using an arbitrary vector $\mathbf{s} \in \mathbb{R}^m$ selected at the initialization stage, we adjust the sign of the columns of $\hat{\mathbf{A}}$ and $\hat{\mathbf{B}}$ as follows

$$\tilde{\mathbf{A}} = \hat{\mathbf{A}} \text{diag}(\text{sign}(\hat{\mathbf{A}}^\top \mathbf{s})),$$

$$\tilde{\mathbf{B}} = \hat{\mathbf{B}} \text{diag}(\text{sign}(\hat{\mathbf{A}}^\top \mathbf{s})).$$

The column order ambiguity is addressed by ordering the singular vectors in the descending order of their respective singular values.

Theorem 1. The optimal momentum step for acceleration in Algorithm 3 after every t epochs is

$$\beta = (1 - \sqrt{1 - \rho_t})^2, \quad (10)$$

where the corresponding t -epoch unaccelerated rate $\rho_t = \rho((\mathbf{I} - \mathbf{Q})^{(n+m)rt})$ with $\mathbf{Q}$ from (6). Furthermore, the expected error is bounded by

$$\|\mathbb{E}[\mathbf{A}_k \mathbf{B}_k^\top - \mathbf{M}]\|_F^2 \leq C\beta^k, \quad (11)$$

for some finite constant $C > 0$ depending only on $\|\mathbf{A}_0 \mathbf{B}_0^\top - \mathbf{M}\|_F^2$ and $\mathbf{M}$.

A sketch of the proof of Theorem 1 can be found in Section IV. The optimal momentum parameter in (10) depends on the t -epoch rate which is defined by the matrix $\mathbf{Q}$ from (6). Since we do not

Algorithm 2: Sign-Aligned Refactor, modified from [11]

Input: Factors $\mathbf{A} \in \mathbb{R}^{m \times r}$ and $\mathbf{B} \in \mathbb{R}^{n \times r}$, Sign direction $\mathbf{s} \in \mathbb{R}^m$
Output: Normalized factors $\tilde{\mathbf{A}} \in \mathbb{R}^{m \times r}$ and $\tilde{\mathbf{B}} \in \mathbb{R}^{n \times r}$

- 1: $\mathbf{G} = (\mathbf{B}^\top \mathbf{B})(\mathbf{A}^\top \mathbf{A}) \quad \triangleright O((m+n+r)r^2)$
- 2: Perform the EVD: $\mathbf{G} = \mathbf{P} \mathbf{\Lambda} \mathbf{P}^{-1} \quad \triangleright O(r^3)$
- 3: $\mathbf{\Sigma} = \mathbf{\Lambda}^{1/2} \quad \triangleright O(r)$
- 4: $\mathbf{D} = (\mathbf{\Sigma}^{-1} \mathbf{P}^\top \mathbf{A}^\top \mathbf{A} \mathbf{P})^{1/2} \quad \triangleright O(r^3)$
- 5: $\mathbf{Q} = \mathbf{D} \mathbf{P}^{-1}, \mathbf{Q}^{-1} = \mathbf{P} \mathbf{D}^{-1} \quad \triangleright O(r^2)$
- 6: $\hat{\mathbf{A}} = \mathbf{A} \mathbf{Q}^{-1} \quad \triangleright O(mr^2)$
- 7: $\hat{\mathbf{B}} = \mathbf{B} \mathbf{Q}^\top \quad \triangleright O(nr^2)$
- 8: $\tilde{\mathbf{A}} = \hat{\mathbf{A}} \text{diag}(\text{sign}(\hat{\mathbf{A}}^\top \mathbf{s})) \quad \triangleright O(r^2 + mr)$
- 9: $\tilde{\mathbf{B}} = \hat{\mathbf{B}} \text{diag}(\text{sign}(\hat{\mathbf{A}}^\top \mathbf{s})) \quad \triangleright O(nr)$

have the solution before the algorithm is executed, we propose to estimate the true rate as a mean of random problems $\mathbf{M}_i$ with the same observation matrix $\mathbf{\Omega}$. This average can be performed on either the eigenvalue, $\mathbb{E}[(1 - \lambda_{\min}(\mathbf{Q}(\mathbf{M}_i, \mathbf{\Omega})))^{(n+m)rt}]$, or the rate, $(\mathbb{E}[1 - \lambda_{\min}(\mathbf{Q}(\mathbf{M}_i, \mathbf{\Omega}))])^{(n+m)rt}$.

Remark 2. As the matrix dimensions increase, our preliminary analysis of random matrix $\mathbf{Q}$ suggests that its eigenvalues approach a limiting distribution with finite support. Moreover, for large matrices this distribution correctly predicts the rate of the algorithm. Further, the support of the limiting eigenvalue distribution of $\mathbf{Q}$ depend only on $\rho_s = \frac{s}{nm}$, $\rho_r = 1 - \sqrt{(1 - \frac{r}{n})(1 - \frac{r}{m})}$, and n/m .

While we maintain that our analysis of the mean error is correct for all $t \geq 1$, we find that in some cases for small matrices and small values of t that the mean is not the only parameter relevant to the rate of convergence of typical sequences. Increasing t allows analysis of the mean to translate to analysis of particular executions of the algorithm.

IV. ERROR ANALYSIS

In this section, we provide a proof sketch for Theorem 1 in Section III. We begin with an argument that the direct update on the factors $\mathbf{A}_k$ and $\mathbf{B}_k$ leads to an asymptotically similar update on $\mathbf{X}_k = \mathbf{A}_k \mathbf{B}_k^\top$:

$$\mathbf{X}_{k+1} = \tilde{\mathbf{X}}_k + \beta(\mathbf{X}_k - \mathbf{X}_{k-1}) + O(\|\mathbf{X}_k - \mathbf{X}_*\|_F^2). \quad (12)$$

This follows from the expansion of $\mathbf{A}_k, \mathbf{B}_k$, and $\mathbf{X}_k$ around their fixed points: $\mathbf{A}_k = \mathbf{A}_* + \Delta \mathbf{A}_k$, $\mathbf{B}_k = \mathbf{B}_* + \Delta \mathbf{B}_k$, and $\mathbf{X}_k = \mathbf{X}_* + \Delta \mathbf{X}_k$. Further perturbation analysis is left to the reader due to space constraints.

By projecting the error $\mathbf{X}_k - \mathbf{X}_*$ and taking the expectation as in (7), we can define $\mathbf{v}_k = \mathbb{E}[\mathbf{G}^{1/2} \mathbf{Z}^\top (\mathbf{X}_k - \mathbf{M})]$ and obtain

$$\mathbf{v}_{k+1} = (\mathbf{I} - \mathbf{Q})^{((m+n)rt)} \mathbf{v}_k + \beta(\mathbf{v}_k - \mathbf{v}_{k-1}) + O(\|\mathbf{v}_k\|^2),$$

where we can replace $\mathbf{X}_*$ with the explicit form of the minimizer $\mathbf{M}$. This equation takes the form of (5) from Section II-A where $\mathbf{T} = (\mathbf{I} - \mathbf{Q})^{(n+m)rt}$ and $\mathbf{q}_k$ is $O(\|\mathbf{v}_k\|^2)$ so by selecting

$$\beta = (1 - \sqrt{1 - \rho_t})^2$$

$\mathbf{v}_k$ to converge at the rate of $\sqrt{\beta}$. Since this transformation $\mathbf{G}^{1/2} \mathbf{Z}^\top$ is full rank, equality of norms leads to the bound in Theorem 1.

Remark 3. If we expect $\rho_t = (1 - \delta)^t$ for $\delta \ll 1$, then $\rho_t \approx 1 - t\delta$, and the expected number of epochs needed to reach a precision $\bar{\epsilon}$ is proportional to $\frac{t}{-\log(\sqrt{\beta})} \approx \sqrt{t/\delta}$. By contrast, the unaccelerated algorithm requires the number of epochs proportional to $1/\delta$.

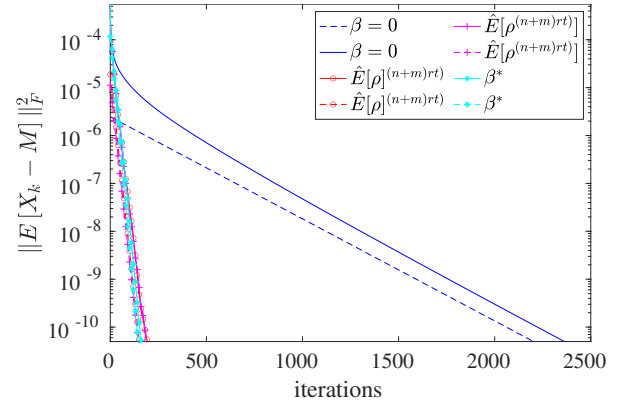
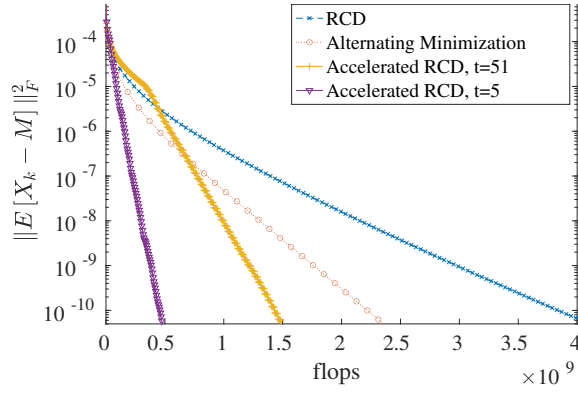


Fig. 1: Squared Frobenius error of mean distance for an 80×80 matrix. (Left) vs. flops for different algorithms. The algorithms shown are the unaccelerated RCD (*), optimally accelerated RCD (+ and ∇), and alternating minimization ($\circ$). (Right) vs. the number of iterations for $\beta = 0$ (blue) for β estimated in the two ways described ($\circ$ and +), and for β^* (*). The dashed straight lines are the predicted asymptotic rate, and the solid lines are the results generated from executing the algorithms.

Algorithm 3: Momentum Accelerated RCD for Matrix Completion

Input: $\mathcal{P}_\Omega(M)$, β , A_0 , B_0 , t

Output: $\{A, B\}$

- 1: Initialize $A_c \leftarrow A_0$, $A_o \leftarrow A_0$ and $B_c \leftarrow B_0$, $B_o \leftarrow B_0$
 - 2: Choose random $s \in \mathbb{R}^m$
 - 3: $A_n \leftarrow A_c$, $B_n \leftarrow B_c$
 - 4: **for** $l = 1, 2, \dots$ **do**
 - 5: **for** $i = 1, 2, \dots, t$ **do**
 - 6: $(A_n, B_n) \leftarrow \text{RCD}(A_n, B_n, s)$ $\triangleright O(sr)$
 - 7: $A_n \leftarrow A_n + \beta(A_c - A_o)$ $\triangleright O(mr)$
 - 8: $B_n \leftarrow B_n + \beta(B_c - B_o)$ $\triangleright O(nr)$
 - 9: $A_o \leftarrow A_c$, $B_o \leftarrow B_c$
 - 10: $A_c \leftarrow A_n$, $B_c \leftarrow B_n$
 - 11: $A = A_c$, $B = B_c$
-

V. NUMERICAL RESULTS

We produced simulations to show our method to be efficient compared to the state of the art and that an appropriate β can be obtained without using the solution.

1) *Experimental settings:* We selected $s = e_1^{(m)}$ to produce these figures. The figures use an 80 by 80 matrix M generated with $\rho_r = 0.15$ and $\rho_s = 0.29$ and the initial values A^0 and B^0 were generated from the true factors with random additive noise. The acceleration step was performed every t epochs as marked in Fig. 1-(left) and every $t = 5$ epochs in Fig. 1-(right). Each figure shows the error from sample means from 5 runs from the same initialization.

Fig. 1-(left) shows the performance of the optimal β^* calculated from the true Q for the problem for each value of t compared in the number of flops to the performance of the unaccelerated version and to alternating minimization. We considered alternating minimization due to its computational efficiency [9], [19], and selected an implementation that minimizes each of the rows of each factor separately and is $O(sr^2)$. The Lightspeed Matlab toolbox¹ was used to track the number of flops for each operation of each algorithm; we used r^3 to be the exact number of flops for the eigenvalue decomposition, but this is not the dominant term.

¹The toolbox can be found at Tom Minka's Github: <https://github.com/tminka/lightspeed>

Fig. 1-(right) uses β calculated based on the rate estimated by 10 random solution matrices using the same Ω , with both estimation methods suggested. It also compares these to β^* from the true Q in terms of number of iterations, each of which is t epochs. The asymptotic rates were produced from the spectrum of the true Q based on the β selected from the estimates.

2) *Results:* In Fig. 1-(left) we see that the accelerated algorithm is the most computationally efficient algorithm and outperforms alternating minimization even when the unaccelerated method does not due to the small size of the problem. In addition, the figure demonstrates that various values of t allow convergence, and increasing values of t yield smoother convergence. Fig. 1-(right) instead shows that exact knowledge of ρ_t is not required to achieve performance gains with momentum since the accelerated versions achieve a faster rate than the unaccelerated version (blue) despite generating the value of the momentum parameter without reference to the rate of the unaccelerated algorithm. In addition, the rates in Fig. 1-(right) approach the asymptotic bounds support our theory of the asymptotic rate.

VI. CONCLUSION

In this paper, we present an acceleration method for randomized coordinate descent matrix completion. We avoid increasing the computational complexity via an epoch-level acceleration step. We present a convergence analysis of the proposed algorithm resulting in a closed-form expression for the asymptotic linear convergence rate. This leads to a closed-form expression for the optimal momentum parameter. Numerical experiments establish the correctness of the rate expression and establish the effectiveness of the acceleration method. In future work, we would like to further obtain an expression for the rate of convergence that can be expressed in terms independent of the solution, such as the ratio of known entries to the matrix size, the shape of the matrix, and the relative rank. This would ease determination of the optimal momentum parameter. In addition, this would allow an a priori prediction of the total number of flops required to achieve a certain accuracy, allowing for a comprehensive comparison between methods for matrix completion.

REFERENCES

- [1] Nathan Srebro and Tommi Jaakkola, “Weighted low-rank approximations,” in *Proc. Int. Conf. Mach. Learn.*, 2003, pp. 720–727.
- [2] Ji Liu, Przemyslaw Musialski, Peter Wonka, and Jieping Ye, “Tensor completion for estimating missing values in visual data,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 1, pp. 208–220, 2013.
- [3] Luong Trung Nguyen, Junhan Kim, Sangtae Kim, and Byonghyo Shim, “Localization of iot networks via low-rank matrix completion,” *IEEE Trans. Commun.*, vol. 67, no. 8, pp. 5833–5847, 2019.
- [4] Cong Ma, Kaizheng Wang, Yuejie Chi, and Yuxin Chen, “Implicit regularization in nonconvex statistical estimation: Gradient descent converges linearly for phase retrieval and matrix completion,” in *Proc. Int. Conf. Mach. Learn.* PMLR, 2018, pp. 3345–3354.
- [5] Trung Vu and Raviv Raich, “On the asymptotic linear convergence of gradient descent for non-symmetric matrix completion,” in *Asilomar Conf. Signals Syst. Comput.* IEEE, 2023, pp. 1049–1053.
- [6] Tian Tong, Cong Ma, and Yuejie Chi, “Accelerating ill-conditioned low-rank matrix estimation via scaled gradient descent,” *Journal of Machine Learning Research*, vol. 22, no. 150, pp. 1–63, 2021.
- [7] Lijun Ding and Yudong Chen, “Leave-one-out approach for matrix completion: Primal and dual analysis,” *IEEE Trans. Inf. Theory*, vol. 66, no. 11, pp. 7274–7301, 2020.
- [8] Trung Vu, Evgenia Chumikhina, and Raviv Raich, “On local linear convergence rate of iterative hard thresholding for matrix completion,” *IEEE Trans. Signal Process.*, vol. 70, pp. 5940–5953, 2022.
- [9] Prateek Jain, Praneeth Netrapalli, and Sujay Sanghavi, “Low-rank matrix completion using alternating minimization,” in *Proc. Annu. ACM Symp. Theory Comput.*, 2013, pp. 665–674.
- [10] Moritz Hardt, “Understanding alternating minimization for matrix completion,” in *Proc. Annu. IEEE Symp. Found. Comput. Sci.*, 2014, pp. 651–660.
- [11] Matthew Callahan, Trung Vu, and Raviv Raich, “Provable randomized coordinate descent for matrix completion,” in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.* IEEE, 2024, pp. 9421–9425.
- [12] Yurii Nesterov, “Efficiency of coordinate descent methods on huge-scale optimization problems,” *SIAM J. Optim.*, vol. 22, no. 2, pp. 341–362, 2012.
- [13] Filip Hanzely and Peter Richtárik, “Accelerated coordinate descent with arbitrary sampling and best rates for minibatches,” in *Proc. Int. Conf. Artif. Intell. Stat.* 2019, pp. 304–312, PMLR.
- [14] Qin Wang, Weiguo Li, Wendi Bao, and Feiyu Zhang, “Accelerated randomized coordinate descent for solving linear systems,” *Mathematics*, vol. 10, no. 22, pp. 4379, 2022.
- [15] David G Luenberger and Yinyu Ye, *Appendix A*, p. 507–514, Springer, 3 edition, 2008.
- [16] Trung Vu and Raviv Raich, “A closed-form bound on the asymptotic linear convergence of iterative methods via fixed point analysis,” *Optim. Lett.*, vol. 1, pp. 1–14, 2022.
- [17] Boris T Polyak, *Multistep Methods*, pp. 65–67, New York, Optimization Software, 1987.
- [18] Robert M Gower and Peter Richtárik, “Randomized iterative methods for linear systems,” *SIAM J. Matrix Anal. Appl.*, vol. 36, no. 4, pp. 1660–1690, 2015.
- [19] Dongha Lee, Jinoh Oh, and Hwanjo Yu, “OCAM: Out-of-core coordinate descent algorithm for matrix completion,” *Inf. Sci.*, vol. 514, pp. 587–604, 2020.